🎤 ROTEIRO DE APRESENTAÇÃO EM SALA — ROTA VITAL

⏱ 15 a 20 Minutos

Guia Oficial de Falas e Demonstração — Unidade 2 — Desafios PI (Parte 1)

Projeto: Banco de Sangue Inteligente | **Tempo Total:** 15 a 20 minutos | **Equipe:** 5 Apresentadores

1. Cronograma Geral e Divisão de Blocos

Bloco	Apresentador	Tempo	Tópico Central da Fala
1	**Eduardo Borges**	3 a 4 min	Abertura, propósito do sistema, preservação da U1 e Arquitetura Hexagonal (perfis `u1` vs `infra`).
2	**Luiz Henrique Rocha**	3 a 4 min	E1: Dockerfile multi-stage sem skip de testes, usuário non-root e orquestração Docker Compose.
3	**Eliziane Mota**	3 a 4 min	E2: Ciclo TDD Red-Green (H1, H2, H3), chave serial `id_seq` no Flyway e subquery atômica LIFO.
4	**Pedro Iranildo**	3 a 4 min	E2: Pirâmide de testes (Surefire vs Failsafe + Testcontainers) e persistência pós-reinício (`BolsaPersistidaIT`).
5	**Ricardo Severiano**	4 a 5 min	E3: Pipeline DAG no GitHub Actions, deploy com commit SHA no Render, rollback e demonstração ao vivo.

👤 Bloco 1: Eduardo Borges

DESENVOLVEDOR BACKEND

Tempo Estimado: 3 a 4 min

🖥️ Slide Recomendado: Diagrama da Arquitetura Hexagonal e Transição Pedagógica (U1 → U2)

1. Abertura e Propósito do Sistema:

> "Boa noite, professor e colegas. Nosso grupo é formado por Eduardo, Luiz Henrique, Eliziane, Pedro e Ricardo. Estamos apresentando a entrega da Unidade 2 de Infraestrutura de Software do projeto **Rota Vital**. O Rota Vital é um sistema de gestão de banco de sangue voltado ao controle de estoque de bolsas, atendimento emergencial de requisições hospitalares e triagem com detecção de duplicatas."

2. A Transição Pedagógica da U1 para a U2:

> "Na Unidade 1, trabalhamos com estruturas de dados manuais com nós dinâmicos em C e em Java — lista encadeada para estoque e fila FIFO para requisições —, além de threads para cálculo de similaridade Levenshtein. Na Unidade 2, recebemos a missão de elevar esse backend para o padrão industrial: containerizar a solução, persistir em PostgreSQL real, adotar TDD rigoroso e automatizar o ciclo completo de CI/CD e deploy."

3. Arquitetura Hexagonal Mínima e Estratégia de Perfis:

> "O desafio central foi: como persistir no banco sem descartar ou corromper as estruturas da U1? Resolvemos isso com o padrão de **Portas e Adaptadores**. Criamos a interface `EstoqueStorage`: sob o perfil `u1`, ativamos o `ListaEstoqueStorage` (100% em memória com a lista manual); sob o perfil `infra`, ativamos o `JdbcEstoqueStorage` com PostgreSQL 16 e Flyway. E uma regra inegociável da equipe: **proibição estrita de fallback silencioso**. Se o banco falhar sob o perfil infra, a aplicação falha de forma ruidosa e reporta status DOWN no Actuator, sem mascarar falhas na memória."

👤 Bloco 2: Luiz Henrique Rocha

TECH LEADER

Tempo Estimado: 3 a 4 min

🖥️ Slide Recomendado: Trechos do Dockerfile Multi-Stage e compose.yaml com Healthchecks

1. Dockerfile Multi-Stage e Eliminação do Skip de Testes:

> "Passando para a containerização (E1), reformulamos integralmente o Dockerfile. No legado da U1, existia a má prática da flag `-DskipTests`, que ignorava testes na compilação. Nós eliminamos isso: no estágio de builder, executamos `./mvnw package -DskipITs -q`. Qualquer teste unitário que quebre interrompe o build da imagem imediatamente na origem."

2. Segurança Non-Root e Controle de Processos:

> "No estágio de runtime, usamos a imagem enxuta `eclipse-temurin:21-jre-alpine`. Criamos um usuário e grupo dedicados sem privilégios (`rotavital:rotavital`, UID 10001). Nunca rodamos como root. Além disso, o entrypoint invoca formalmente `exec java $JAVA_OPTS -jar app.jar`. O comando `exec` faz com que a JVM assuma o PID 1 no contêiner, respondendo corretamente a sinais POSIX como `SIGTERM` para encerramento gracioso."

3. Orquestração com Docker Compose:

> "No `compose.yaml`, orquestramos os serviços `api` e `db` (PostgreSQL 16). O banco possui healthcheck nativo com `pg_isready` a cada 5s, e a API usa `depends_on: db: condition: service_healthy`, subindo somente quando o banco estiver aceitando conexões TCP. A persistência é assegurada pelo volume nomeado `rotavital-postgres-data`, com comunicação na rede bridge isolada `rotavital-network` e segredos blindados via `.env.example`."

👤 Bloco 3: Eliziane Mota

QA ENGINEER

Tempo Estimado: 3 a 4 min

🖥️ Slide Recomendado: Tabela de Commits Red → Green e DDL da Migração V1 com Subquery LIFO

1. Disciplina do Ciclo Red → Green → Refactor:

> "No bloco E2, a persistência do estoque seguiu estritamente a disciplina do TDD. Não escrevemos código de produção sem antes termos um teste vermelho falhando pelo motivo esperado:
> • **H1 (Cadastro/Recuperação):** falha provocada pela ausência de classes e DTO (commit `2525767`), seguida pela implementação e aprovação (commits `07c44a7` e `8aa3432`).
> • **H2 (Listagem LIFO):** falha pela falta do método `listar` (commit `d5157f5`), seguida pelo retorno ordenado e envelope JSON (commits `1a1589f` e `38dd729`).
> • **H3 (Remoção):** 5 testes vermelhos com `UnsupportedOperationException` (commit `2b07ecc`), seguida pela exclusão atômica no banco (commit `9a4b69b`)."

2. O Desafio dos IDs Repetidos e a Subquery LIFO:

> "A lista encadeada da U1 permitia cadastrar bolsas com o mesmo ID e buscava/removia sempre a mais recente (LIFO). Para reproduzir isso no PostgreSQL, criamos na migração Flyway uma chave técnica autoincrementável: `id_seq BIGSERIAL PRIMARY KEY`, mantendo o campo de negócio `id` sem restrição UNIQUE.
> Para a remoção em H3, executamos a subquery atômica:
> `DELETE FROM bolsas WHERE id_seq = (SELECT id_seq FROM bolsas WHERE id = ? ORDER BY id_seq DESC LIMIT 1)`.
> Essa consulta apaga cirurgicamente a ocorrência mais recente, deixando as mais antigas intactas, com custo O(1) graças ao índice composto `idx_bolsas_id_seq`."

👤 Bloco 4: Pedro Iranildo

DEVOPS ENGINEER

Tempo Estimado: 3 a 4 min

🖥️ Slide Recomendado: Pirâmide de Testes (Surefire vs Failsafe) e Código da BolsaPersistidaIT

1. Pirâmide de Testes e Segregação no Maven:

> "Organizamos nossos testes em três níveis bem segregados no `pom.xml`:
> • **Nível 1 (Unitários rápidos via Surefire):** 201 testes que executam via `mvnw test` em apenas 22 segundos, sem dependência de banco ou Docker.
> • **Nível 2 e 3 (Integração e Aceitação via Failsafe + Testcontainers):** executam via `mvnw -Pintegration verify`. Sobem contêineres reais de PostgreSQL 16 para validar migrações Flyway, transações e os endpoints REST com MockMvc."

2. Comprovação de Sobrevivência Pós-Reinício (`BolsaPersistidaIT`):

> "O teste definitivo de persistência é a classe `BolsaPersistidaIT`. O teste grava a bolsa no PostgreSQL real, instancia um segundo contexto Spring totalmente novo a frio conectado ao mesmo banco e atesta que todos os campos (ID, tipo sanguíneo, volume, validade e doador) sobrevivem 100% íntegros. Fizemos o mesmo para comprovar a ordem LIFO e a remoção definitiva pós-reinício."

3. Blindagem de Não-Regressão da U1:

> "Preservamos 100% do patrimônio da U1 sem enfraquecer nenhum critério: mantemos aprovadas as **54 asserções em C** com AddressSanitizer e UBSan (zero vazamentos de memória), os **7 testes Python** e a consistência estatística de todas as 40 medições históricas de speedup."

👤 Bloco 5: Ricardo Severiano de Souza Filho

PRODUCT OWNER

Tempo Estimado: 4 a 5 min

🖥️ Slide Recomendado: DAG do GitHub Actions, Proteção de Branch e Tela da Aplicação no Ar

1. Esteira CI/CD em DAG com 6 Gates (GitHub Actions):

> "No bloco E3, estruturamos o pipeline como um Grafo Acíclico Dirigido (DAG) com 6 gates encadeados:
> • Gates 1, 2 e 3 rodam em paralelo: SO em Python, AED em C e Java unitário Surefire.
> • Gate 4: Testes de integração com Testcontainers PostgreSQL 16.
> • Gate 5: Build da imagem Docker e Smoke Test containerizado de 9 etapas contra a API e o banco.
> • Gate 6: Deploy automatizado no Render.
> Zero contornos permissivos: não existe `continue-on-error` nem `|| true`. Se um teste quebrar, a esteira aborta imediatamente."

2. Deploy com Commit SHA e Governança de Rollback:

> "O deploy no Render só é disparado se todos os 5 gates anteriores passarem na branch `main`, repassando o commit SHA validado: `curl -X POST '${RENDER_DEPLOY_HOOK_URL}?ref=${{ github.sha }}'`.
> Estabelecemos rollback em 3 vias (por hook com SHA anterior, pelo painel do Render e via `git revert`) e protegemos o banco com a estratégia Expand/Contract e `spring.flyway.clean-disabled=true`.
> Além disso, o **Pull Request #1** foi homologado e mesclado na `main`, e ativamos a **Proteção de Branch** exigindo os status checks obrigatórios antes de qualquer merge."

3. Demonstração Prática ao Vivo e Conclusão:

> "Para concluir, convidamos a banca a verificar a aplicação no ar em **https://rota-vital.onrender.com**:
> • `/actuator/health` → Retorna status UP com PostgreSQL ativo.
> • `/api/estoque` → Retorna HTTP 200 com envelope padronizado.
> Entregamos um sistema robusto, auditado e com paridade industrial. Estamos prontos para as perguntas da banca. Obrigado!"

2. Guia de Comandos para Demonstração ao Vivo no Terminal

```
# 1. Executar 201 testes unitários Java (Surefire ~22s):
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-25.0.4.101-hotspot"
.\mvnw.cmd test

# 2. Executar testes em C com AddressSanitizer e UBSan (54 asserções sem vazamento):
mingw32-make -C aed-c test

# 3. Executar benchmarks e validação estatística de SO em Python:
python -m unittest discover -s bench -p 'test_*.py'
python bench/plot_speedup.py --check

# 4. Subir ambiente completo local via Docker Compose (API + PostgreSQL):
docker compose up -d --build
docker compose ps

# 5. Testar a aplicação em produção via cURL:
curl -i https://rota-vital.onrender.com/actuator/health
curl -i https://rota-vital.onrender.com/api/estoque
```

3. FAQ — Perguntas Prováveis da Banca e Respostas Técnicas

P1: Por que usaram Spring Data JDBC em vez de JPA / Hibernate?

Resposta (Eduardo / Eliziane): Porque o domínio da Release 1 é delimitado à entidade `Bolsa`. O JDBC puro nos dá controle determinístico sobre o SQL executado, permitindo escrever a subquery atômica LIFO sem overhead de proxies, cache de primeiro nível ou risco de `LazyInitializationException`, inicializando em milissegundos e consumindo muito menos memória no Alpine.

P2: Como garantem que a remoção de duplicatas apaga a ocorrência certa sob concorrência?

Resposta (Eliziane / Eduardo): A remoção é atômica no PostgreSQL via subquery: `DELETE FROM bolsas WHERE id_seq = (SELECT id_seq FROM bolsas WHERE id = ? ORDER BY id_seq DESC LIMIT 1)`. Como o PostgreSQL executa a subquery dentro da transação com lock de linha na escrita, requisições concorrentes apagam sequencialmente a mais recente e a penúltima, sem leituras sujas.

P3: Como o Dockerfile garante que nenhum teste quebrado vá para a imagem final?

Resposta (Luiz Henrique): No estágio builder, removemos deliberadamente a flag `-DskipTests` e executamos `./mvnw package -DskipITs -q`. Se qualquer teste quebrar, o Maven retorna código 1 e o Docker interrompe o build imediatamente. O runtime usa usuário non-root `rotavital` e consome `exec java $JAVA_OPTS -jar` para assumir o PID 1 e responder a `SIGTERM`.

P4: Como comprovaram que a branch main está protegida contra pushes diretos?

Resposta (Ricardo / Pedro): A proteção está formalmente ativada e verificada via GitHub REST API (`"protected": true`). Configuramos regras estritas exigindo aprovação prévia dos 5 status checks do CI (SO, AED C, Java Unit, Java Integration e Docker Smoke) antes de qualquer merge.

P5: Por que a esteira de CI/CD foi estruturada em DAG com 6 gates?

Resposta (Ricardo / Luiz Henrique): Para aplicar os princípios de Fail-Fast e paralelismo: Python, C e Java unitário rodam simultaneamente no início. Somente se passarem, o Gate 4 inicia o Testcontainers com banco real. Se passar, o Gate 5 constrói a imagem Docker e roda o Smoke Test com Compose. O Gate 6 (Deploy no Render) só executa se todos os 5 gates anteriores passarem na branch `main`.